10/04

Quick Sort

Quick also follows divide and conquer method

In Quick sort we select one of the element as pivot(key)
we need to place this pivot in its best place , so that elements
On left of the pivot are smaller than pivot , elements on right
of the pivot should be greater than pivot.

How to select the pivot value:
any element of the an array can be pivot, but most of the time
we consider
1. left element
2. right most element
3.middle element
as pivot

If pivot is dividing the array into equal parts in all cases
then it leads to best case with time complexity as O(nlogn)
other wise it leads worst case with time complexity as $O(n^2)$

to implement the Quick sort we take 'l' and 'r'.
the task 'l' is to identify the value greater than pivot on left si
the task of 'r' is to identity the value samller than pivot on
right side
if 'l' is samller than 'r' swap the values between the 'l' and 'r'
index
else 'r' is the best place for pivotso swap between r and pivot
repeat this process recursively until we get a array with sin
element

p = a[low] = 4

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
4	~~8~~ 3	1	2	9	~~3~~ 8	10

l=0 l=1 → swap → r=5 r=6

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
~~4~~ 2	3	1	~~2~~ 4	9	8	10

l=1 l=2 l=3 l=4 r=5 r=4 r=3

as l is not smaller than r so swap with pivot

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]
2	3	1	4	9	8	10

p=2

a[0]	a[1]	a[2]
2	~~3~~ 1	~~1~~ 3

l=0 l=1 r=2

a[0]	a[1]	a[2]
~~2~~	~~1~~ 2	3

l=1 r=2 r=1 l=2

a[0]	a[1]	a[2]
1	2	3

a[0] = 1 a[1] = 3

p = a[4] = 9

a[4]	a[5]	a[6]
~~8~~ 9	~~2~~ 9	10

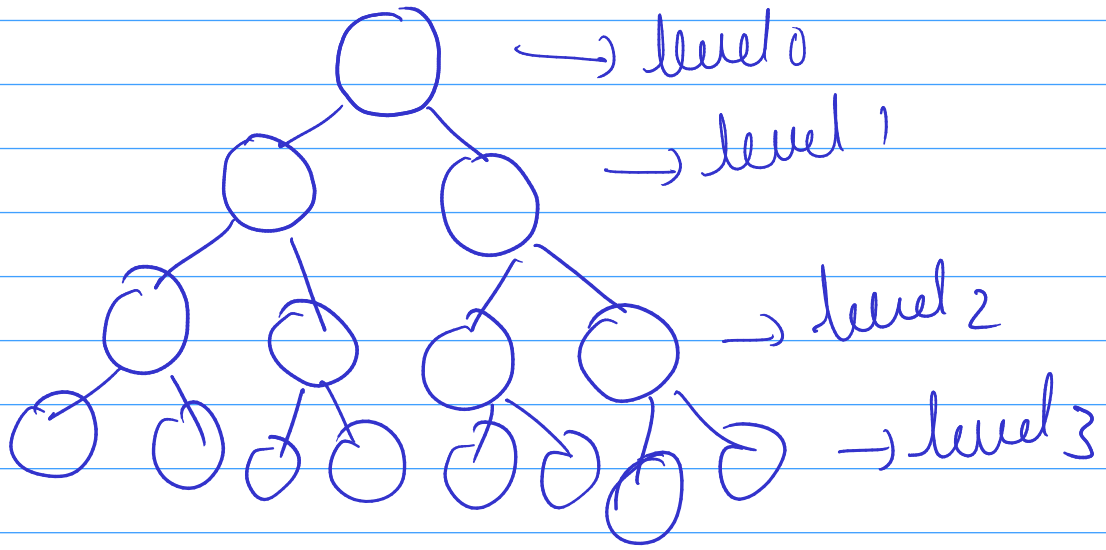
l=9 l=5 r=6 r=5 l=6

a[4]	a[5]	a[6]
8	9	10

a[4] = 8 a[6] = 10

Binary Tree

In a Binary Tree every node can have utmost two children.



If 'L' is the level of a binary tree then maximum nodes 'N' possible at that level is

$N = 2^L$

Ex: if l is 3

$N = 2^3 = 8$

if l is 0

$N = 2^0 = 1$

If 'H' is the height of the binary tree then maximum nodes 'N' possible for that tree is

$N = 2^{H+1} - 1$

Ex: if H is 3 $\quad 2^{3+1} - 1 = 2^4 - 1 = 15$

if 'N' is no of nodes in a binary tree, then its height 'H' is

$N = 2^{H+1} - 1$

$2^{H+1} - 1 = N$

$2^{H+1} = N+1$

Apply $\log_2$ on both sides

$\log_2 2^{H+1} = \log_2 (N+1)$

$H+1 \log_2 2 = \log_2 (N+1)$

$H+1 = \log_2 (N+1)$

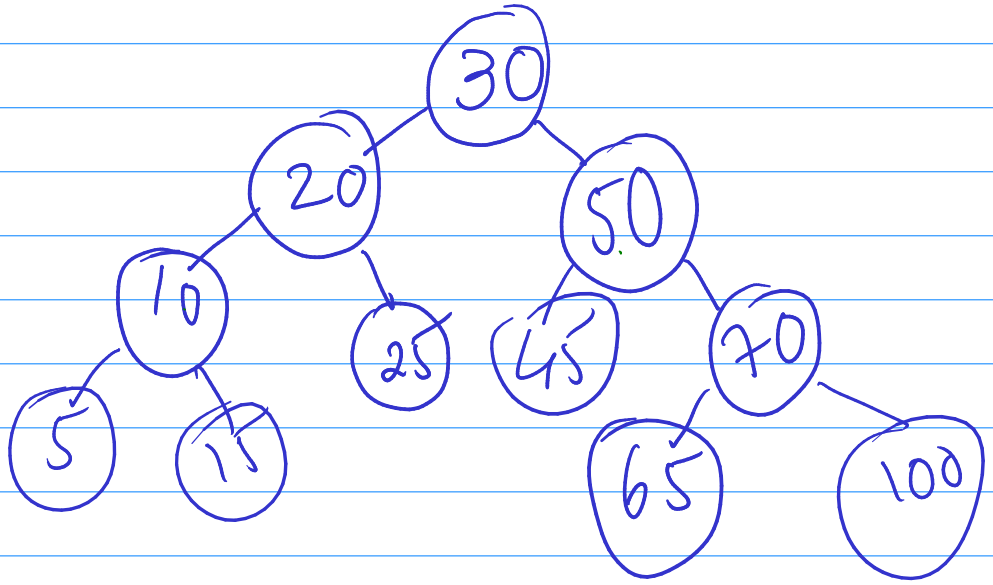
$H = \log_2 (N+1) - 1 \Rightarrow H = \log_2 (N+1)$

$$H = \log_2 N$$

We have 3 traversing methods in Binary Tree or Binary Search Tree

preorder root left right
inorder left root right
postorder left right root

30,50,20,10,25,70,65,45,100,5,15



30, 20, 10, 5, 15, 25, 50, 45, 70, 65, 100

5, 10, 15, 20, 25, 30, 45, 50, 65, 70, 100

5, 15, 10, 25, 20, 45, 65, 100, 70, 50, 30